



CATÁLOGO

Algoritmos de Aprendizaje Supervisado

De la regresión lineal a los *Foundation Models*

Machine Learning Supervisado

El Mapa Completo

~30 algoritmos, de OLS a los Foundation Models

Una ficha consistente por algoritmo: cuándo brilla, cuándo falla, y con qué librería usarlo.

Cómo leer este catálogo

Cada algoritmo tiene una lámina con la misma estructura:

-  Intuición + su frontera de decisión.
-  Pros /  Contrás.
-  Cuándo usar /  cuándo evitar.
-  Condiciones (escalado, tamaño, supuestos).
-  Librería + código ·  docs ·  ejemplo en el curso.
-  Ficha: complejidad · creador/año · requisitos.

Leyenda de *chips*

Familia

Interpretab. ●●○

¿Escalar? Sí/No

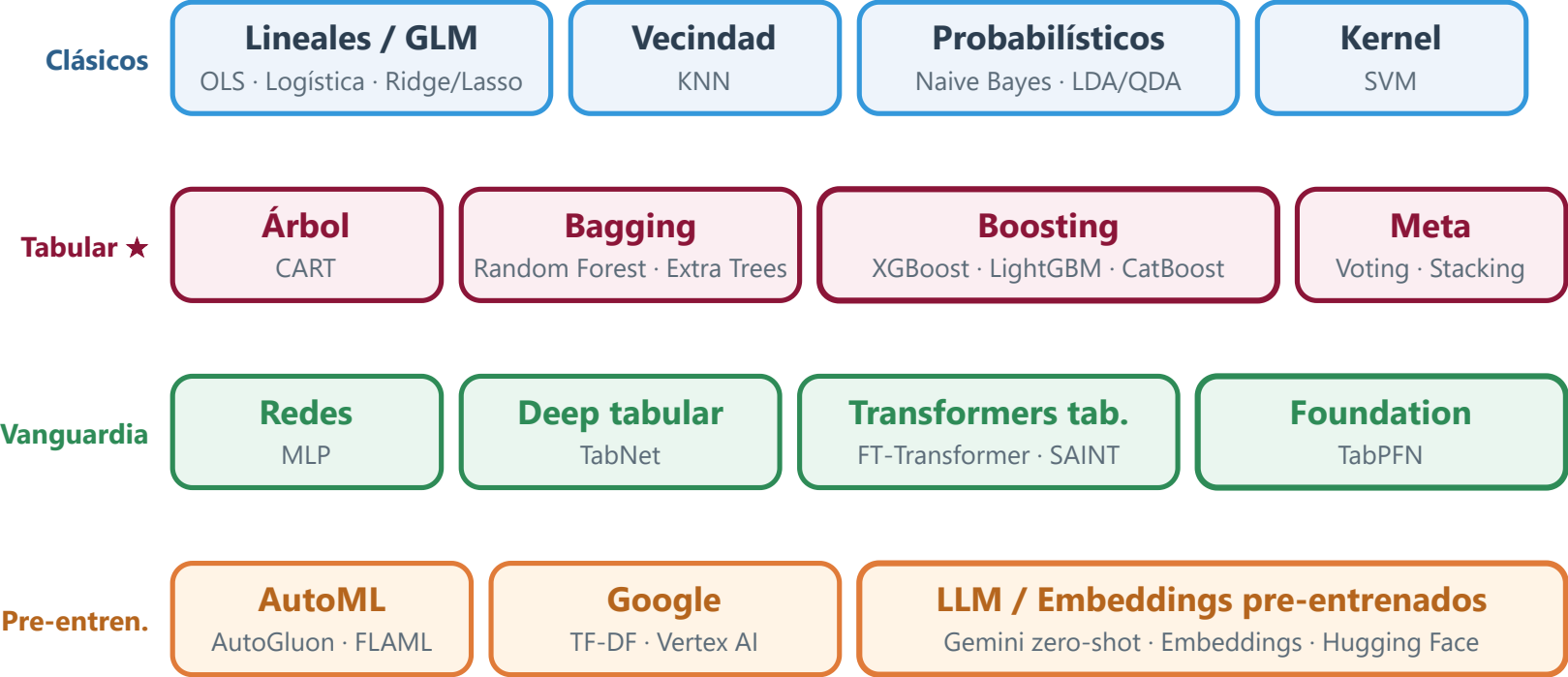
SOTA tabular

Foundation

Cloud / pre-entrenado

 Núcleo del curso ·  tarjetas Ampliación
(vanguardia) son referencia, no se dictan todas en vivo.

Las familias de algoritmos



De arriba (interpretables, baseline) hacia abajo (potencia, estado del arte, sin entrenar)

Ningún algoritmo es "el mejor": cada familia hace un **supuesto distinto** sobre la forma de los datos. El arte está en **elegir y comparar**.

 Tabla maestra (1/2) — clásicos y ensembles

Algoritmo	Familia	Interpr.	No-lineal	¿Escalar?	Datos	Velocidad	Librería
Regresión Lineal	Lineal/GLM	●●●	✗	Sí	S-L	⚡⚡⚡	scikit-learn
Regresión Logística	Lineal/GLM	●●●	✗	Sí	S-L	⚡⚡⚡	scikit-learn
Ridge / Lasso / ElasticNet	Lineal/GLM	●●●	✗	Sí	S-L	⚡⚡⚡	scikit-learn
GLM (Poisson/Tweedie)	Lineal/GLM	●●●	~	Sí	S-L	⚡⚡⚡	sklearn / statsmodels
KNN	Vecindad	●●○	✓	Sí	S-M	⚡ (predice lento)	scikit-learn
Naive Bayes	Probabilístico	●●○	~	No	S-L	⚡⚡⚡	scikit-learn
LDA / QDA	Discriminante	●●○	QDA sí	Sí	S-M	⚡⚡⚡	scikit-learn
SVM (RBF)	Kernel	●○○	✓	Sí	S-M	⚡ (lento >50k)	scikit-learn
Árbol de Decisión	Árbol	●●●	✓	No	S-L	⚡⚡	scikit-learn
Random Forest	Bagging	●●○	✓	No	S-L	⚡⚡	scikit-learn
Extra Trees	Bagging	●●○	✓	No	S-L	⚡⚡⚡	scikit-learn
AdaBoost	Boosting	●●○	✓	No	S-M	⚡⚡	scikit-learn
Gradient Boosting	Boosting	●●○	✓	No	S-M	⚡	scikit-learn
HistGradientBoosting	Boosting	●●○	✓	No	M-L	⚡⚡⚡	scikit-learn
XGBoost	Boosting	●●○	✓	No	M-L	⚡⚡	xgboost
LightGBM	Boosting	●●○	✓	No	L-XL	⚡⚡⚡	lightgbm
CatBoost	Boosting	●●○	✓	No	M-L	⚡⚡	catboost

Datos: S=pequeño · M=mediano · L=grande · XL=masivo

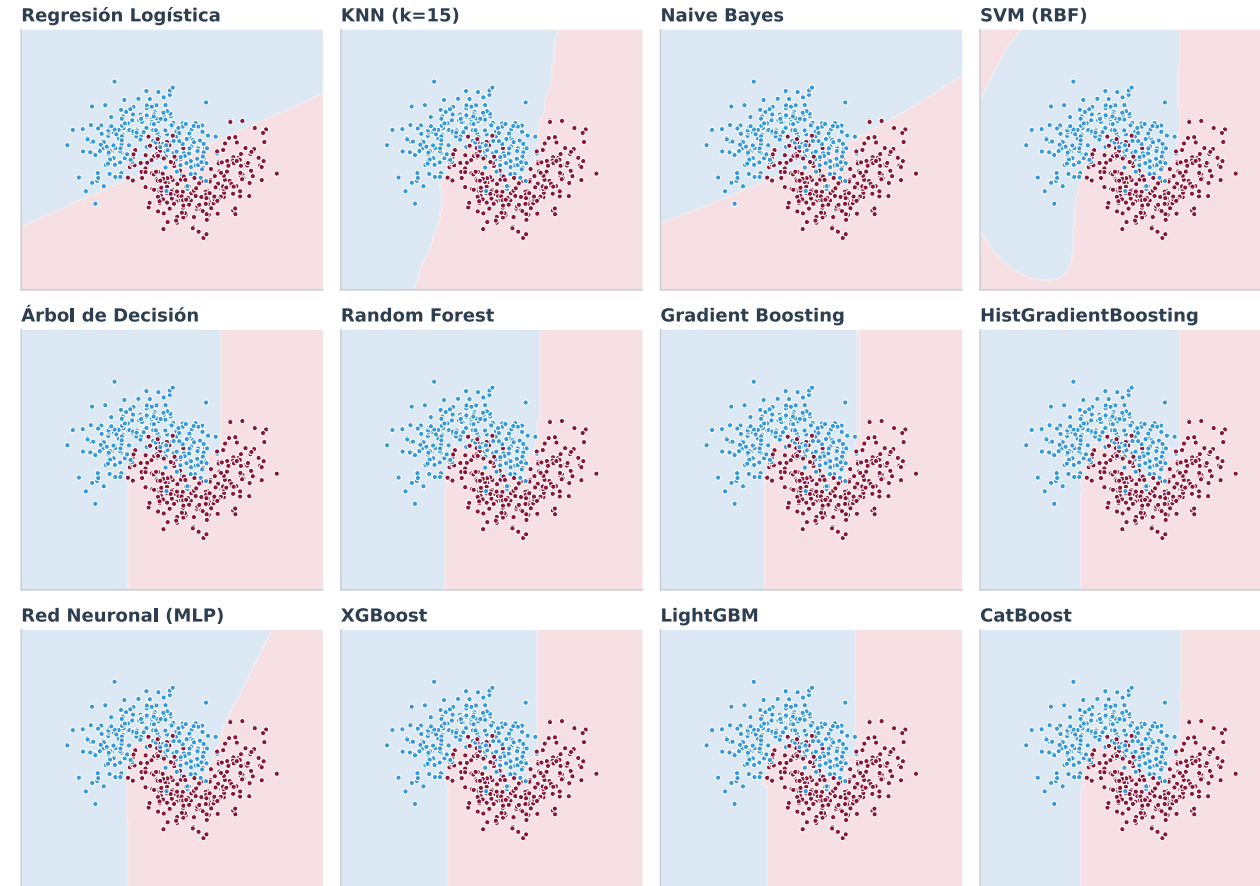
 Tabla maestra (2/2) — vanguardia y pre-entrenados

Algoritmo	Familia	Interpr.	No-lineal	¿Escalar?	Datos	Velocidad	Librería
NGBoost	Boosting probab.	●●○	✓	No	S–M	⚡	ngboost
Voting / Stacking	Meta-ensemble	●○○	✓	depende	S–L	⚡	scikit-learn
MLP (red neuronal)	Redes	●○○	✓	Sí	M–L	⚡	sklearn / keras
TabNet	Deep tabular	●●○ (atención)	✓	Sí	L–XL	🔌 (GPU)	pytorch-tabnet
FT-Transformer / SAINT	Transformer tabular	●○○	✓	Sí	L–XL	🔌 (GPU)	pytorch / rtdl
TabPFN	Foundation tabular	●○○	✓	Sí	S (<10k)	⚡ (sin entrenar)	tabPFN
AutoGluon / FLAML	AutoML	varía	✓	auto	M–XL	🔌 (busca solo)	autogluon / flaml
TF Decision Forests	Árboles (Google)	●●○	✓	No	M–XL	⚡⚡	tensorflow_decision_forests
Vertex AI AutoML	Cloud (Google)	●○○	✓	auto	L–XL	☁ managed	google-cloud-aiplatform
Gemini (zero/few-shot)	LLM pre-entrenado	●○○	✓	—	sin datos	☁ API	google-genai
Embeddings + clasificador	Pre-entrenado	●○○	✓	—	texto	☁/local	google-genai / sentence-transformers
Hugging Face zero-shot	Pre-entrenado	●○○	✓	—	texto	☁/local	transformers



Misma data, distinta frontera

Misma data, distinta frontera: cada algoritmo 've' el problema a su manera

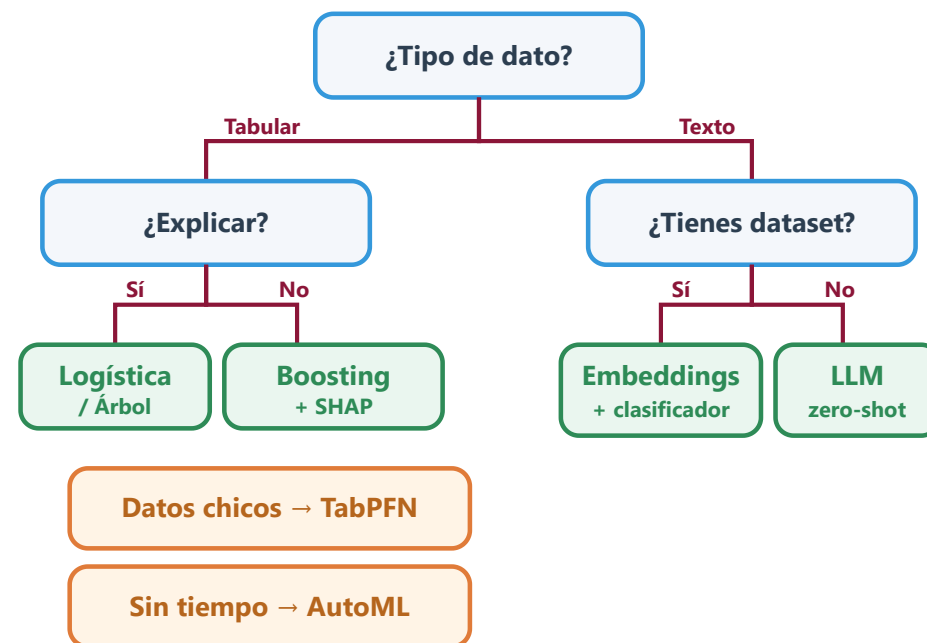


El mismo problema (clasificar dos lunas) resuelto por 12 algoritmos. La forma de la frontera revela el supuesto de cada uno: recta (logística), curvas suaves (SVM/boosting), escalonada (árbol), local (KNN).

🧭 ¿Qué algoritmo elijo?

Receta práctica

1. **Baseline:** Dummy + Regresión Logística.
2. **¿Datos tabulares?** → **boosting**
(LightGBM/XGBoost/CatBoost). Casi siempre gana.
3. **¿Necesitas explicar?** → Logística / Árbol, o boosting + SHAP.
4. **¿Dataset chico (<10k) y quieres SOTA sin tunear?**
→ TabPFN.
5. **¿Sin tiempo para tunear?** → AutoML
(AutoGluon/FLAML).
6. **¿Texto / pocos ejemplos?** → embeddings o LLM zero-shot (Gemini/HF).



Siempre: empieza por un baseline (Dummy + Logística)

1 Modelos Lineales y GLM

Interpretables · el baseline a vencer

Regresión Lineal (OLS)

Lineal / GLM

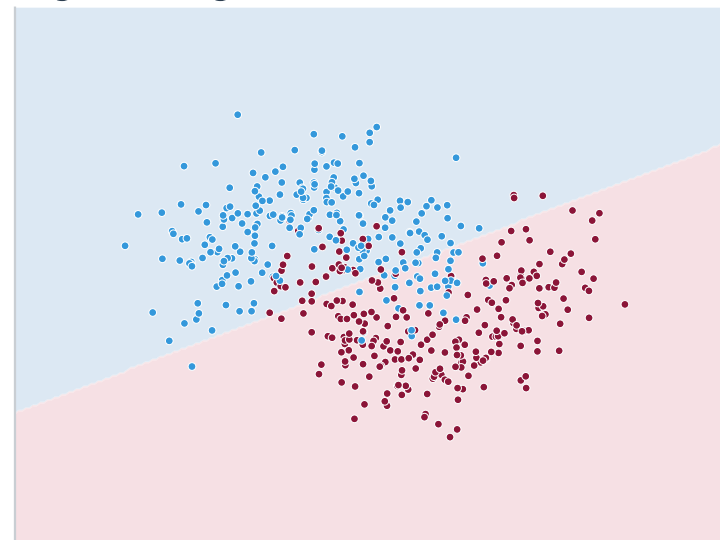
Interpretab. ●●●

¿Escalar? Sí

Regresión

- 🧠 Ajusta la **recta (hiperplano)** que minimiza el error cuadrático.
- ✅ **Pros:** transparente · rapidísima · coeficientes = efecto directo.
- ❌ **Contras:** solo relaciones lineales · sensible a outliers y multicolinealidad.
- 🎯 **Usar:** relación ~lineal, necesitas explicar el efecto de cada variable.
- 🚫 **Evitar:** patrones no lineales o muchas interacciones.

Regresión Logística



⚠️ **Condiciones:** linealidad, homocedasticidad, normalidad de residuos, no multicolinealidad ($VIF < 5$) ·  `from sklearn.linear_model import LinearRegression` ·  scikit-learn.org · 🕒 **Ejemplo:** M01/00_Regresion_Lineal_Intro · 📊 $O(n \cdot p^2)$ · Legendre/Gauss ~1805

Regresión Logística

Lineal / GLM

Interpretab. ●●●

¿Escalar? Sí

Baseline de oro

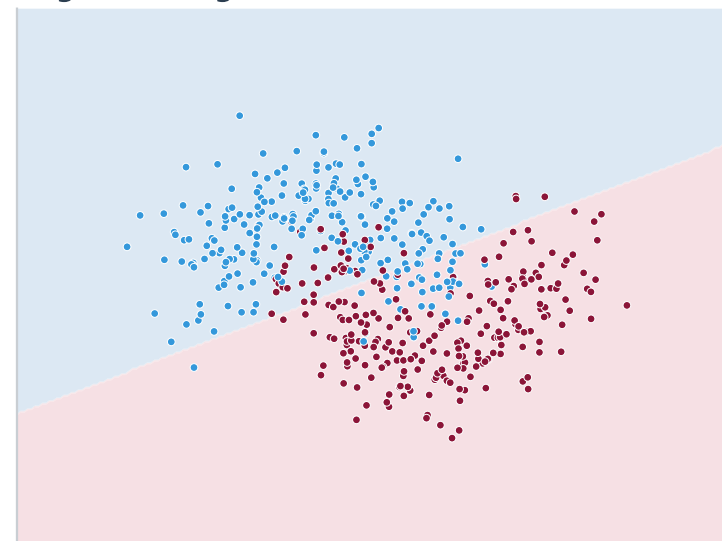
🧠 Aplica la **sigmoide** a una combinación lineal → **probabilidad** $[0,1]$.

✅ **Pros:** probabilidades calibradas · Odds Ratio interpretable · estándar regulatorio (banca, salud).

❌ **Contras:** frontera lineal en el log-odds · no captura interacciones solo.

🎯 **Usar:** clasificación con necesidad de **explicar** (scoring, riesgo).

🚫 **Evitar:** relaciones muy no lineales (usa árboles/boosting).

Regresión Logística

⚠️ **Condiciones:** linealidad en el log-odds, independencia, ≥ 10 eventos por variable ·  `LogisticRegression(class_weight='balanced')` ·  scikit-learn.org ·  **Ejemplo:** M01/00_Regresion_Logistica_Intro ·  $O(n \cdot p)$ · Cox 1958

⚖ Ridge · Lasso · ElasticNet

Lineal / GLM

Interpretab. ●●●

¿Escalar? Sí

Regularizados

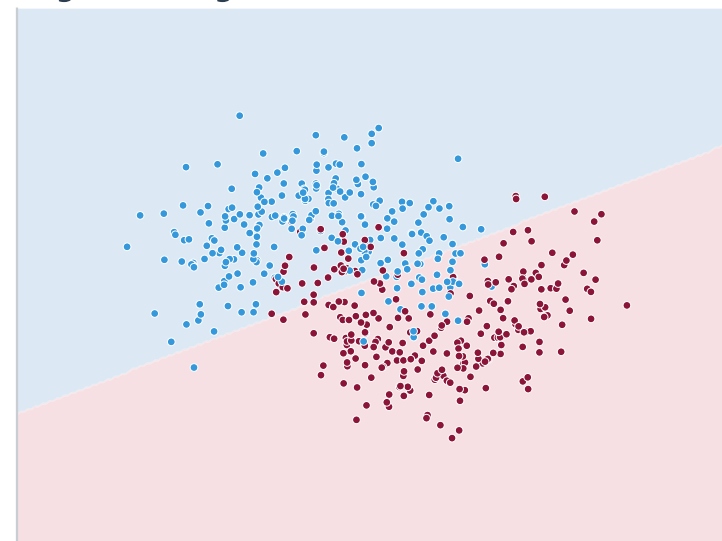
🧠 Regresión **penalizada**: Ridge (L2) **encoge** coeficientes; Lasso (L1) los lleva a **cero** (selección); ElasticNet combina.

✅ **Pros**: controla overfitting y multicolinealidad · Lasso = selección automática de variables.

❌ **Contras**: sigue siendo lineal · hay que tunear λ (C).

🎯 **Usar**: muchas variables correlacionadas, pocas filas.

🚫 **Evitar**: cuando necesitas no-linealidad fuerte.

Regresión Logística

⚠ **Condiciones**: escalar SIEMPRE (la penalización depende de la escala) · 💻 `LogisticRegression(penalty='l1', C=0.5, solver='saga')` · 🌐 scikit-learn.org · 📄 **Ejemplo:** M01/02_Pipelines · 📖 Hoerl 1970 (Ridge), Tibshirani 1996 (Lasso)

GLM — Poisson / Tweedie

Lineal / GLM

Interpretab. ●●●

Ampliación

 Generaliza la regresión lineal a **otras distribuciones**: Poisson para **conteos**, Tweedie para montos con muchos ceros (seguros).

✓ **Pros**: modela el target con su distribución real · interpretable · estándar actuarial.

✗ **Contras**: requiere elegir la familia/función de enlace · sigue lineal en el predictor.

🎯 **Usar**: frecuencia de siniestros, demanda, conteos.

🚫 **Evitar**: clasificación binaria estándar (usa logística).



conteos / montos
→ distribución propia
(Poisson · Gamma · Tweedie)

⚠ **Condiciones**: elegir familia y enlace acordes al target ·  `from sklearn.linear_model import PoissonRegressor, TweedieRegressor` ·  scikit-learn.org ·  statsmodels.org ·  Nelder & Wedderburn 1972

2 Vecindad, Probabilísticos y Kernel

🔍 KNN — K-Nearest Neighbors

Vecindad

Interpretab. ●●○

¿Escalar? Sí

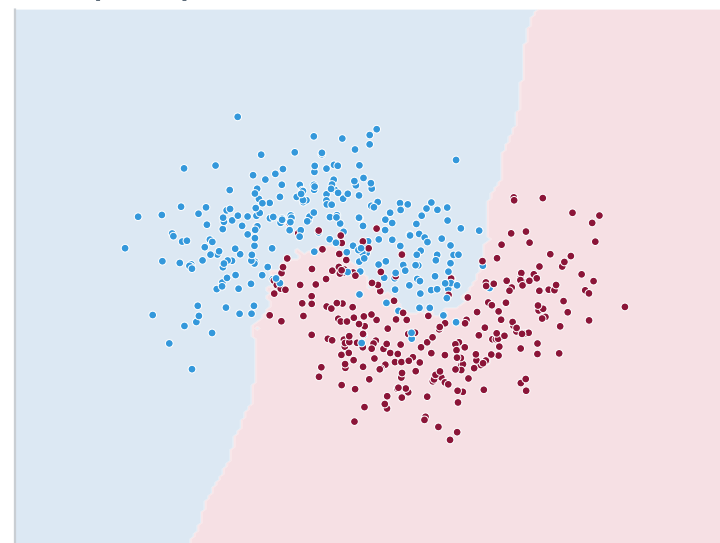
🧠 "*Dime con quién andas*": clasifica según la **clase mayoritaria** de los K vecinos más cercanos.

✅ **Pros:** sin entrenamiento · no paramétrico · captura fronteras locales complejas.

❌ **Contras:** **predicción lenta** (recorre todo) · maldición de la dimensionalidad · sensible a escala.

🎯 **Usar:** datasets chicos, imputación (`KNNImputer`), recomendación.

🚫 **Evitar:** muchas filas/columnas o predicción en tiempo real.

KNN (k=15)

⚠️ **Condiciones:** escalar SIEMPRE (usa distancias) · elegir K (impar) · `KNeighborsClassifier(n_neighbors=15, weights='distance')` · scikit-learn.org · 🌟 **Ejemplo:** M02/00e_KNN_Intro · 📊 O(n·p) por query · Fix & Hodges 1951

🎮 Naive Bayes

Probabilístico

Interpretab. ●●○

¿Escalar? No

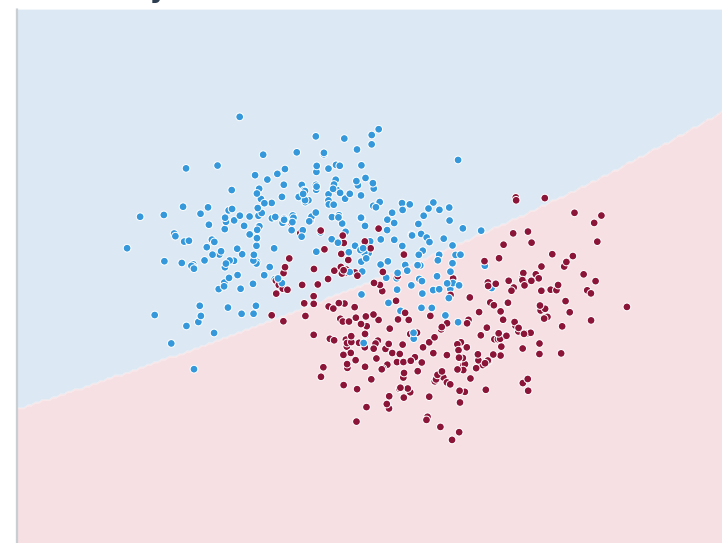
🧠 Aplica el **teorema de Bayes** asumiendo (ingenuamente) que las variables son **independientes** dada la clase.

✅ **Pros:** rapidísimo · funciona con **pocos datos** · excelente para **texto** (spam, sentimiento).

❌ **Contras:** el supuesto de independencia rara vez se cumple · probabilidades mal calibradas.

🎯 **Usar:** clasificación de texto, baseline ultra-rápido, alta dimensión.

🚫 **Evitar:** variables muy correlacionadas donde importa la probabilidad exacta.

Naive Bayes

⚠️ **Condiciones:** elegir variante (Gaussian/Multinomial/Bernoulli) según el tipo de dato · `from sklearn.naive_bayes import GaussianNB, MultinomialNB` · `scikit-learn.org` · `O(n·p)` · siglo XVIII (Bayes)

🔺 LDA / QDA — Análisis Discriminante

Discriminante

Interpretab. ●●○

¿Escalar? Sí

Ampliación

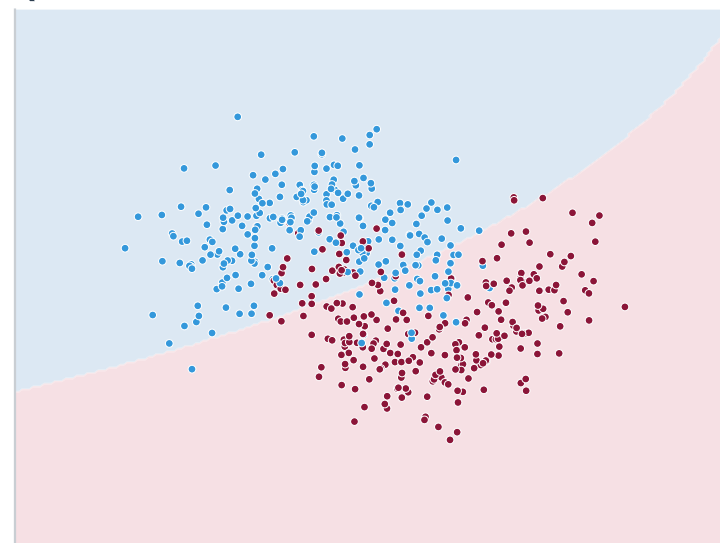
🧠 Modela cada clase como una **gaussiana**. LDA asume misma covarianza (frontera **lineal**); QDA permite distinta (frontera **curva**).

✅ **Pros:** rápido · LDA sirve también para **reducción de dimensión** · buen baseline.

❌ **Contras:** asume normalidad · QDA necesita más datos por clase.

🎯 **Usar:** clases ~gaussianas, reducción supervisada de dimensión.

🚫 **Evitar:** distribuciones muy no normales.

QDA

⚠️ **Condiciones:** normalidad por clase · QDA: suficientes casos por clase · 💻 `from sklearn.discriminant_analysis import LinearDiscriminantAnalysis` · 📦 scikit-learn.org · 🇺🇸 Fisher 1936

SVM — Support Vector Machine

Kernel

Interpretab. ●○○

¿Escalar? Sí

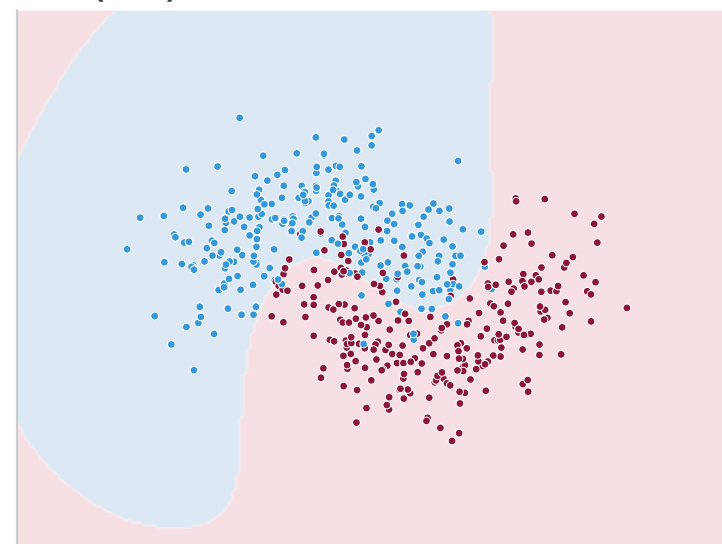
🧠 Busca el **hiperplano de máximo margen**; el *kernel trick* (RBF) lo lleva a dimensiones donde los datos sí se separan.

✅ **Pros:** potente en alta dimensión · márgenes robustos · kernels flexibles.

❌ **Contras:** lento (>50k filas) · sensible a escala · no da probabilidades por defecto.

🎯 **Usar:** datasets chicos/medianos con frontera compleja, alta dimensión (texto, bioinformática).

🚫 **Evitar:** datos masivos (usa boosting).

SVM (RBF)

⚠️ **Condiciones:** escalar SIEMPRE · tunear C y gamma · `SVC(kernel='rbf', C=1, gamma='scale')` · scikit-learn.org · 📄 **Ejemplo:** M02/00d_SVM_Intro · 📊 $O(n^2 - n^3)$ · Cortes & Vapnik 1995

3 Árboles y Ensembles

Los reyes del dato tabular

🌳 Árbol de Decisión (CART)

Árbol

Interpretab. ●●●

¿Escalar? No

🧠 Hace **preguntas sí/no** sobre las variables, partiendo los datos hasta hojas "puras".

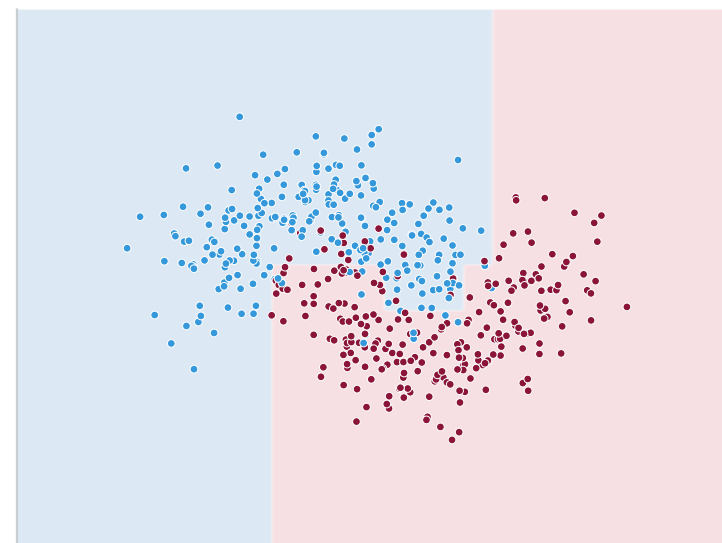
✅ **Pros:** muy interpretable (reglas If-Then) · no requiere escalado · maneja categóricas y no-linealidad.

❌ **Contras:** **overfitting** severo sin poda · frontera escalonada · inestable.

🎯 **Usar:** explicar reglas de negocio, base de los ensembles.

🚫 **Evitar:** como modelo final solo (mejor en bosque/boosting).

Árbol de Decisión



⚠️ **Condiciones:** controlar `max_depth` , `min_samples_leaf` ·  `DecisionTreeClassifier(max_depth=5)` ·  scikit-learn.org · 🌐 **Ejemplo:** M02/00a_Arboles_Decision_Intro ·  Breiman 1984

Random Forest

Bagging

Interpretab. ●●○

¿Escalar? No

Baseline robusto

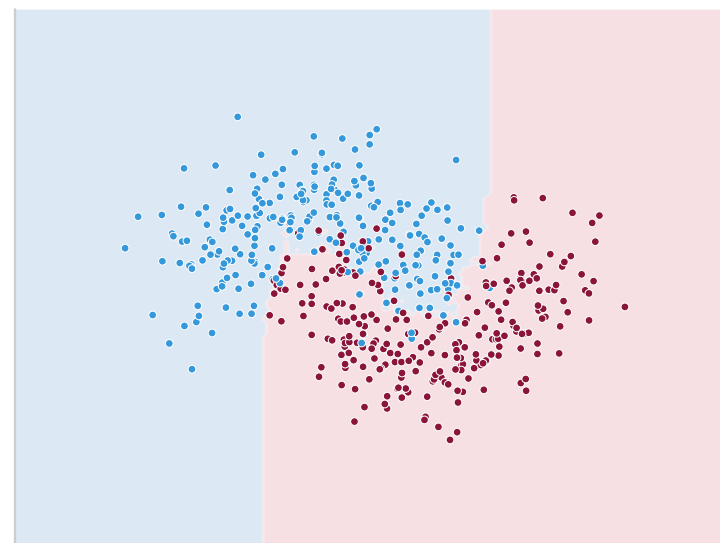
🧠 Promedia **cientos de árboles** entrenados en muestras y variables distintas (*bagging*) → reduce la **varianza**.

✅ **Pros:** robusto y difícil de romper · paraleliza (`n_jobs=-1`) · feature importance · poco tuning.

❌ **Contras:** menos interpretable · modelos grandes en memoria · puede perder vs boosting afinado.

🎯 **Usar:** baseline fuerte en tabular, datasets pequeños/medianos.

🚫 **Evitar:** cuando necesitas el máximo desempeño con datos masivos.

Random Forest

⚠️ **Condiciones:** ninguna especial (invariante a la escala) · `RandomForestClassifier(n_estimators=300, n_jobs=-1)` · scikit-learn.org · 📄 **Ejemplo:** M02/00b_Random_Forest_Intro · 📖 Breiman 2001

Extra Trees (Extremely Randomized)

Bagging

Interpretab. ●●○

¿Escalar? No

Ampliación

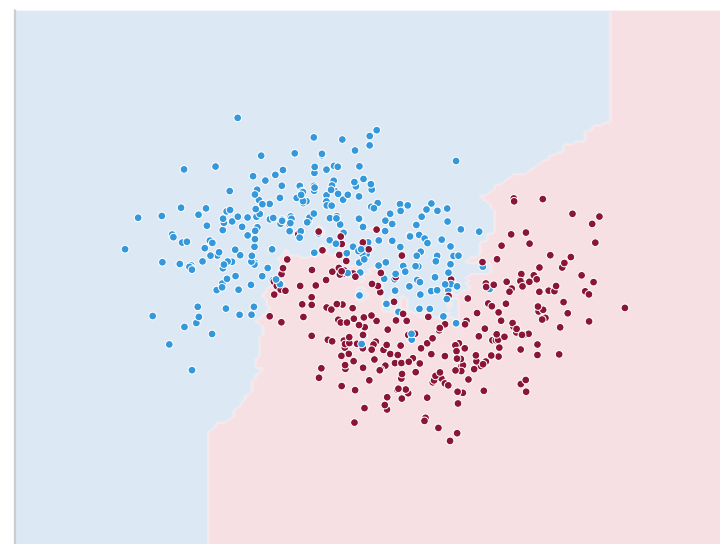
🧠 Como Random Forest pero los **cortes son aleatorios** (no busca el mejor) → más diversidad, más rápido.

✅ **Pros:** más rápido que RF · menor varianza aún · buen desempeño "gratis".

❌ **Contras:** puede tener más sesgo · menos intuitivo el porqué de cada split.

🎯 **Usar:** alternativa veloz a RF, ensembles diversos.

🚫 **Evitar:** cuando necesitas el split "óptimo" interpretable.

Extra Trees

⚠️ **Condiciones:** ninguna especial · `from sklearn.ensemble import ExtraTreesClassifier` ·  scikit-learn.org ·  Geurts et al. 2006

AdaBoost

Boosting

Interpretab. ●●○

¿Escalar? No

Ampliación

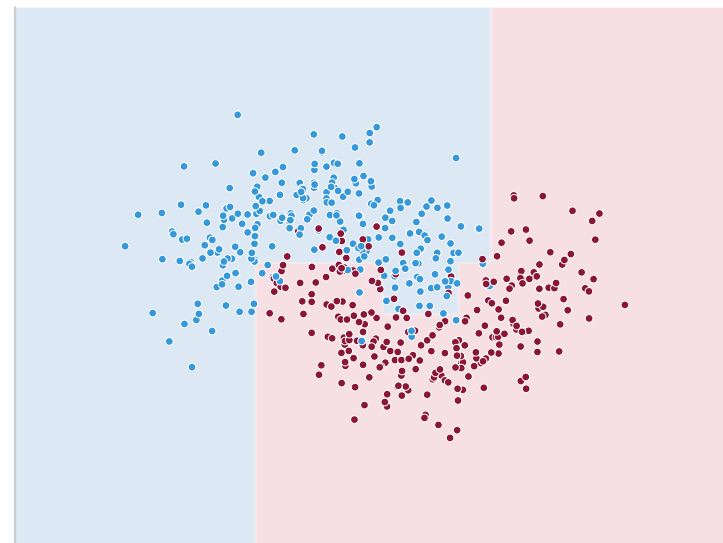
🧠 El **primer boosting**: entrena árboles débiles en **serie**, dando más peso a los ejemplos mal clasificados.

✅ **Pros**: convierte modelos débiles en uno fuerte · pocos hiperparámetros.

❌ **Contras**: sensible a outliers y ruido (los persigue) · superado por GBM moderno.

🎯 **Usar**: entender la idea del boosting, datasets limpios.

🚫 **Evitar**: datos ruidosos (usa Gradient Boosting / LightGBM).

AdaBoost

⚠️ Condiciones: limpiar outliers · `from sklearn.ensemble import AdaBoostClassifier` · scikit-learn.org · [Freund & Schapire 1997 \(Premio Gödel\)](#)

🔥 Gradient Boosting

Boosting

Interpretab. ●●○

¿Escalar? No

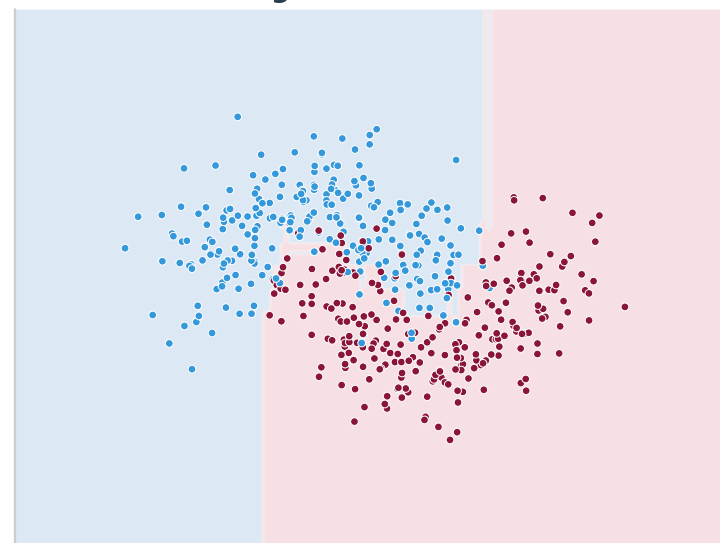
🧠 Cada árbol **corrige los residuos** del anterior, descendiendo por el gradiente de la pérdida.

✅ **Pros:** muy preciso · flexible (cualquier pérdida) · base de XGBoost/LightGBM/CatBoost.

❌ **Contras:** lento (secuencial) · sensible a overfitting sin early stopping · hay que tunear.

🎯 **Usar:** tabular cuando quieres exprimir desempeño y tienes tiempo de tuneo.

🚫 **Evitar:** datasets enormes (usa Hist/LightGBM).

Gradient Boosting

⚠️ **Condiciones:** tunear `learning_rate` + `n_estimators` + early stopping · 💻 `from sklearn.ensemble import GradientBoostingClassifier` · 🌐 scikit-learn.org · 🌟

Ejemplo: M02/00c_Gradient_Boosting_Intro · 📊 Friedman 2001

⚡ HistGradientBoosting

Boosting

Interpretab. ●●○

¿Escalar? No

Rápido (sklearn)

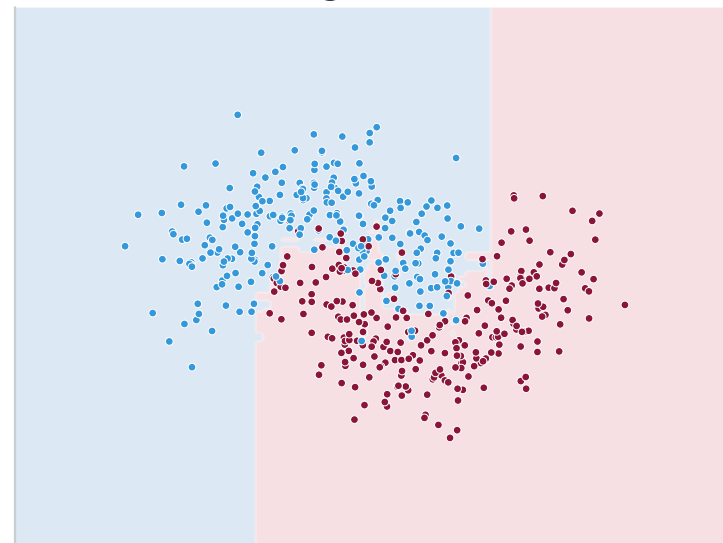
🧠 Gradient Boosting **histogram-based** (inspirado en LightGBM): agrupa valores en *bins* → entrena 10-100× más rápido.

✅ **Pros:** muy rápido · maneja nulos nativamente · soporta categóricas · en sklearn (sin dependencias).

❌ **Contras:** menos *knobs* que XGBoost/LightGBM · más nuevo.

🎯 **Usar:** boosting veloz sin salir de scikit-learn, datos medianos/grandes.

🚫 **Evitar:** cuando ya usas LightGBM/XGBoost afinados.

HistGradientBoosting

⚠️ **Condiciones:** ninguna especial · `from sklearn.ensemble import HistGradientBoostingClassifier` · scikit-learn.org · inspirado en Ke et al. 2017

XGBoost

Boosting

Interpretab. ●●○

¿Escalar? No

SOTA tabular

 Gradient Boosting **regularizado y optimizado**. El rey de Kaggle por años.

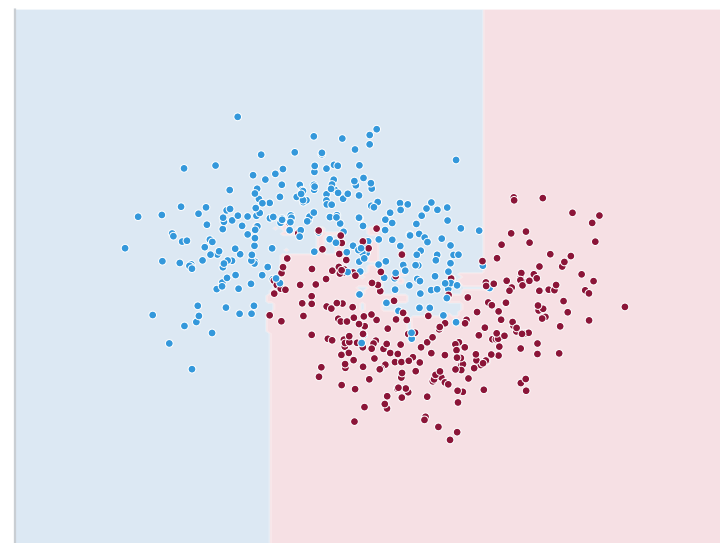
✓ **Pros:** desempeño tope en tabular · regularización L1/L2 · maneja nulos · muy documentado y portable.

✗ **Contras:** muchos hiperparámetros · más lento que LightGBM en datos masivos.

🎯 **Usar:** competencias, producción robusta, tabular en general.

🚫 **Evitar:** datasets XL donde la velocidad manda (LightGBM).

XGBoost



⚠️ **Condiciones:** [tunear con Optuna \(M03\)](#) · `from xgboost import XGBClassifier` · [xgboost.readthedocs.io](#) · 🌟 **Ejemplo:** [M02/02_Arena_Combate](#) · 📊 [Chen & Guestrin 2016 \(KDD\)](#)

⚡ LightGBM

Boosting

Interpretab. ●●○

¿Escalar? No

SOTA · veloz

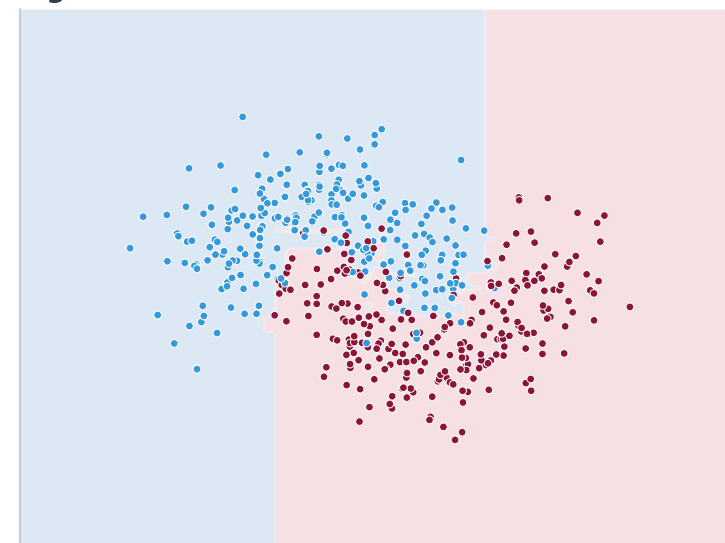
🧠 Boosting de Microsoft con crecimiento **leaf-wise** e histogramas → **velocidad** en datos masivos.

✅ **Pros:** el más rápido · escala a millones de filas · categóricas nativas · gran precisión.

❌ **Contras:** propenso a overfitting en datos chicos · leaf-wise requiere cuidar `num_leaves`.

🎯 **Usar:** datasets grandes, pipelines de producción, scoring (el del curso, M07).

🚫 **Evitar:** datasets muy pequeños sin regularizar.

LightGBM

⚠️ **Condiciones:** controlar `num_leaves`, `min_child_samples`, early stopping · 💻 `from lightgbm import LGBMClassifier` · 📄 lightgbm.readthedocs.io · 🌐 **Ejemplo:** M02/02_Arena, M07 · 📄 Ke et al. 2017 (NeurIPS)

CatBoost

Boosting

Interpretab. ●●○

¿Escalar? No

SOTA · categóricas

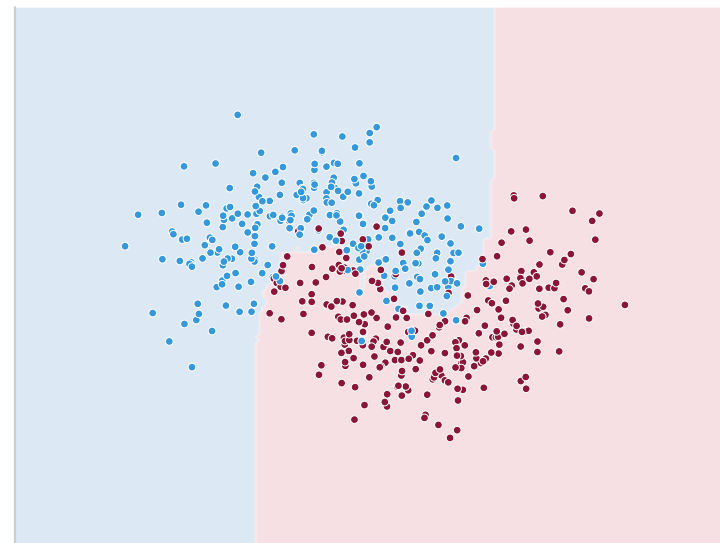
🧠 Boosting de Yandex especializado en **variables categóricas** (las codifica solo, sin leakage) con *ordered boosting*.

✅ **Pros:** excelente con muchas categóricas · **poco tuning** (buenos defaults) · robusto al overfitting.

❌ **Contras:** entrena algo más lento · librería más pesada.

🎯 **Usar:** datos con muchas categóricas de alta cardinalidad, cuando quieres buen resultado sin tunear.

🚫 **Evitar:** cuando ya dominas LightGBM y todo es numérico.

CatBoost

⚠️ Condiciones: pasar `cat_features` · `from catboost import CatBoostClassifier` · `catboost.ai` · Prokhorenkova et al. 2018 (NeurIPS)

NGBoost — Boosting probabilístico

Boosting

Interpretab. ●●○

Predice distribución

Ampliación

🧠 Boosting que predice **toda una distribución** (media + incertidumbre), no solo un punto.

✅ **Pros:** da **intervalos de confianza** nativos · útil cuando la incertidumbre importa (precios, demanda).

❌ **Contras:** más lento · menos maduro que XGBoost/LightGBM.

🎯 **Usar:** cuando necesitas cuantificar **incertidumbre** de cada predicción.

🚫 **Evitar:** si solo te importa el punto y la velocidad.



predice una
distribución
(no solo un número)

⚠️ **Condiciones:** elegir la distribución del target · `from ngboost import NGBClassifier` · github.com/stanfordmlgroup/ngboost ·  Duan et al. 2020 (ICML)

Voting / Stacking

Meta-ensemble

Interpretab. ●○○

¿Escalar? depende

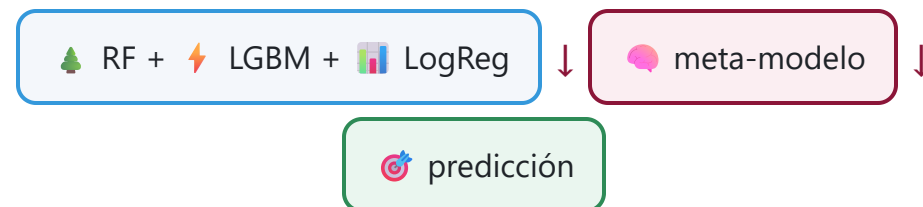
 **Combina varios modelos.** *Voting*: promedia votos. *Stacking*: un meta-modelo aprende a combinar las predicciones de los base.

✓ **Pros:** suele **mejorar** al mejor modelo individual · reduce el riesgo de un solo modelo.

✗ **Contras:** más complejo y lento · difícil de interpretar y mantener.

🎯 **Usar:** exprimir el último % en competencias o producción crítica.

🚫 **Evitar:** cuando la simplicidad y el mantenimiento importan más.



⚠️ **Condiciones:** modelos base diversos · CV para el stacking (evitar leakage) · `from sklearn.ensemble import StackingClassifier, VotingClassifier` · `scikit-learn.org` · 🕒 **Ejemplo:** M03/02_Advanced_Ensembling

4 Redes Neuronales y Deep Tabular

Cuando el dato se vuelve complejo

MLP — Perceptrón Multicapa

Redes neuronales

Interpretab. ●○○

¿Escalar? Sí

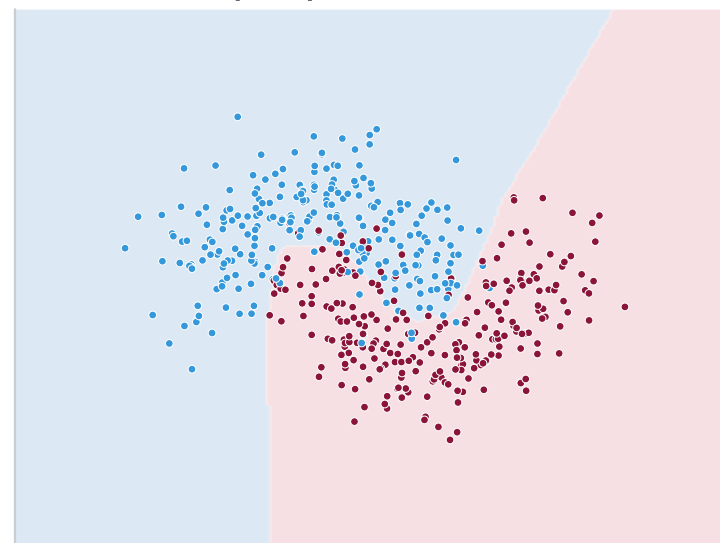
 Capas de neuronas con activaciones no lineales: un **aproximador universal** de funciones.

✓ **Pros:** captura interacciones complejas · base del deep learning · flexible.

✗ **Contras:** en tabular suele perder vs **boosting** · necesita muchos datos, tuning y escalado · caja negra.

🎯 **Usar:** señales complejas, embeddings, como parte de redes mayores.

🚫 **Evitar:** tabular estándar pequeño/mediano (usa boosting).

Red Neuronal (MLP)

⚠ **Condiciones:** escalar SIEMPRE · tunear arquitectura y learning rate · `MLPClassifier(hidden_layer_sizes=(64,32))` / Keras · `scikit-learn.org` · `keras.io` · 🌐 **Ejemplo:** M04 (mención) · 📺 Rumelhart 1986

TabNet — Deep Learning tabular (Google)

Deep tabular

Interpretab. ●●○

Google Research

Requiere GPU

 Red neuronal de **Google** que usa **atención secuencial** para elegir qué variables mirar en cada paso → deep + interpretable.

✓ **Pros:** competitivo en datos grandes · da **importancia de variables** por instancia · aprende representaciones.

✗ **Contras:** necesita **GPU** y muchos datos · más tuning · suele empatar (no superar) al boosting en tabular.

🎯 **Usar:** datasets tabulares **grandes**, cuando quieres deep learning explicable.

🚫 **Evitar:** datasets pequeños/medianos (boosting gana más fácil).



atención: elige
qué variable mirar
en cada paso

⚠ **Condiciones:** GPU recomendada · escalar ·  `from pytorch_tabnet.tab_model import TabNetClassifier` ·  github.com/dreamquark-ai/tabnet ·  Arik & Pfister
(Google) 2019 → AAAI 2021

FT-Transformer / SAINT — Transformers tabulares

Transformer tabular

Interpretab. ●○○

SOTA deep

Requiere GPU

Ampliación

🧠 Llevan la arquitectura **Transformer** (la de los LLM) al dato tabular: tratan cada variable como un "token".

✅ **Pros:** SOTA en deep tabular · capturan interacciones complejas · transfieren conocimiento entre tareas.

❌ **Contras:** GPU + muchos datos · entrenamiento costoso · siguen peleando de igual a igual con el boosting.

🎯 **Usar:** investigación, datasets masivos, cuando el boosting ya tocó techo.

🚫 **Evitar:** la gran mayoría de casos tabulares de negocio (boosting es más simple y barato).



cada columna = "token"
→ **Transformer**
(self-attention)

⚠️ **Condiciones:** GPU, datos grandes, escalado/encoding cuidadoso ·  librería `rtdl` / PyTorch ·  github.com/yandex-research/rtdl ·  Gorishniy et al. 2021 (NeurIPS)

5 Foundation Models y AutoML

El cambio de paradigma

✨ TabPFN — Foundation Model tabular

Foundation tabular

Pre-entrenado

Predice sin entrenar

🧠 Un **transformer pre-entrenado** en millones de datasets sintéticos: le pasas tus datos y **predice en segundos, sin entrenar** (*in-context learning*).

✅ **Pros:** SOTA en tabular **pequeño** (<10k filas) · cero tuning · resultados en **un segundo**.

❌ **Contras:** límite de filas/columnas (v2 amplía) · caja negra · idea muy nueva.

🎯 **Usar:** datasets chicos donde quieres top performance **ya**, sin tunear nada.

🚫 **Evitar:** datasets masivos (todavía).



pre-entrenado →
predice sin entrenar
(in-context learning)

⚠️ **Condiciones:** dataset pequeño · `from tabpfn import TabPFNClassifier` · github.com/PriorLabs/TabPFN · [Hollmann et al. 2023 \(ICLR\)](#) → 2025 (Nature)

AutoML — AutoGluon / FLAML / auto-sklearn

AutoML

Automatiza todo

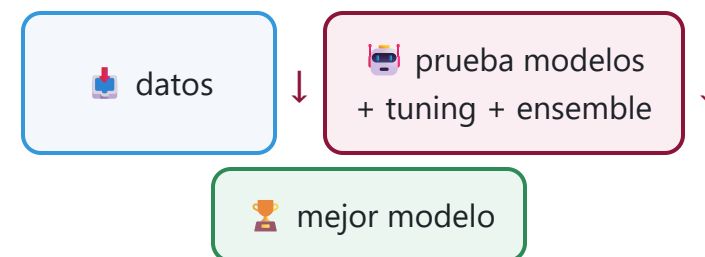
 **Automatiza** el pipeline: prueba modelos, hiperparámetros y *ensembles* por ti, y entrega el mejor.

✓ **Pros:** resultados fuertes con **una línea** · gran baseline · democratiza el ML.

✗ **Contras:** caja negra · puede ser **lento/costoso** · menos control fino · riesgo de no entender el modelo.

🎯 **Usar:** prototipo veloz, baseline difícil de superar, equipos sin tiempo de tuning.

🚫 **Evitar:** cuando necesitas control total o latencia mínima en producción.



⚠ **Condiciones:** presupuesto de tiempo/cómputo · `from autogluon.tabular import TabularPredictor` · auto.gluon.ai · github.com/microsoft/FLAML ·  Erickson et al. 2020

6 Pre-entrenados de Google y la nube

Modelos que ya "saben" — el factor sorpresa

TensorFlow Decision Forests (Google)

Árboles (Google)

TF / Keras

¿Escalar? No

Ampliación

 Implementación de **Google** (Yggdrasil) de Random Forest y Gradient Boosting **dentro de Keras/TensorFlow**.

✓ **Pros:** boosting/RF de calidad producción · se integra con pipelines TF · sirve en TF-Serving · maneja categóricas.

✗ **Contras:** atado al ecosistema TF · curva si no usas Keras.

🎯 **Usar:** ya trabajas en TensorFlow y quieres árboles de producción servibles.

🚫 **Evitar:** si tu stack es sklearn puro (usa LightGBM/XGBoost).



Random Forest / GBT
dentro de Keras
(Google · Yggdrasil)

⚠ **Condiciones:** entorno TensorFlow · `import tensorflow_decision_forests as tfdf` · tensorflow.org/ddecision_forests ·  Google 2021

Vertex AI — AutoML Tabular (Google Cloud)

Cloud (Google)

Managed · sin código

Ampliación

 Servicio **gestionado** de Google Cloud: subes la tabla, eliges el target y Vertex **entrena, evalúa y despliega** el modelo por ti.

✓ **Pros:** sin infraestructura · MLOps integrado (despliegue, monitoreo) · escala a datos enormes · *Model Garden* con modelos pre-entrenados.

✗ **Contras:** costo \$\$ · *vendor lock-in* · menos control y transparencia.

🎯 **Usar:** empresa sin equipo de ML/infra, alto volumen, SLA crítico.

🚫 **Evitar:** presupuesto ajustado o necesidad de control total.


subes la tabla →
Google **entrena +**
despliega por ti

⚠️ **Condiciones:** cuenta de Google Cloud ·  `from google.cloud import aiplatform` ·  cloud.google.com/vertex-ai ·  Google Cloud

Gemini — Clasificación zero/few-shot (LLM)

LLM pre-entrenado

Google · API

Sin datos de entrenamiento

 El LLM de Google clasifica con solo **describir las clases** en el *prompt* (zero-shot) o dar unos ejemplos (few-shot). **No entrenas nada.**

✓ **Pros:** cero datos etiquetados · listo en minutos · razona sobre texto, entiende contexto y matices.

✗ **Contras:** costo por llamada · latencia · puede "alucinar" · menos control · no para tabular numérico puro.

🎯 **Usar:** clasificar **texto** (tickets, reseñas, intención) sin dataset, prototipos rápidos.

🚫 **Evitar:** alto volumen con presupuesto bajo, o scoring numérico regulado.


describes las clases
en el prompt →
clasifica sin entrenar

⚠️ **Condiciones:** API key · diseñar el prompt · validar salidas · `from google import genai` · `ai.google.dev` · Google DeepMind

abc Embeddings pre-entrenados + clasificador

Pre-entrenado

Google / HF

Texto → vector

Ampliación

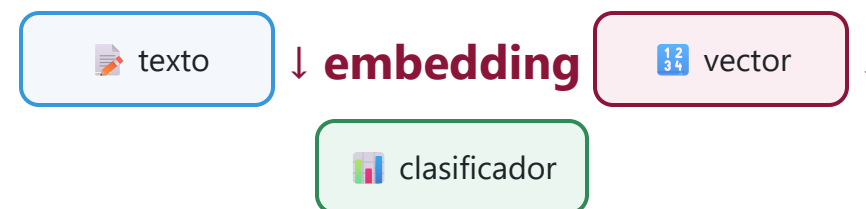
🧠 Un modelo pre-entrenado (Gemini embeddings, Universal Sentence Encoder, BERT) convierte **texto** → **vector**; encima pones un clasificador clásico (logística, LightGBM).

✅ **Pros:** aprovecha conocimiento de millones de textos · clasificador simple, rápido y explicable · barato en inferencia.

❌ **Contras:** dos pasos · el embedding fija el "techo" de calidad · dependes del modelo base.

🎯 **Usar:** clasificar texto con **dataset mediano**, búsqueda semántica, deduplicación.

🚫 **Evitar:** datos puramente numéricos.



⚠️ **Condiciones:** elegir el modelo de embeddings acorde al idioma/dominio · `genai.embed_content(...)` / `sentence-transformers` · `ai.google.dev` · `sbnet.net` · Reimers & Gurevych 2019

🤗 Hugging Face — Zero-shot pipeline

Pre-entrenado

Open source

Texto

Ampliación

🧠 Modelos pre-entrenados (BART-MNLI, etc.) que clasifican texto en **etiquetas que tú defines**, sin entrenar — open source y local.

✅ **Pros:** miles de modelos listos · corre **local** (privacidad) · cero datos etiquetados · gratis.

❌ **Contras:** modelos pesados · más lento en CPU · calidad variable por idioma.

🎯 **Usar:** clasificación de texto privada/offline, prototipos, idiomas con modelo disponible.

🚫 **Evitar:** tabular numérico, o sin GPU para modelos grandes.



pipeline("zero-shot")
+ tus etiquetas →
clasifica local

⚠️ **Condiciones:** elegir modelo por idioma/tarea · `from transformers import pipeline` · huggingface.co · Hub abierto (Meta/Google/...)

🏁 Receta práctica: qué probar primero

Para **dato tabular** (lo más común):

1. 🏆 **Baseline:** Dummy + Regresión Logística.
2. 📈 **Boosting:** LightGBM / XGBoost / CatBoost → casi siempre el mejor.
3. 🔍 **Explica** con SHAP (M04) y calibra (M03).
4. 🧪 Si quieres SOTA sin tunear y los datos son chicos → **TabPFN**.
5. ⚙️ Sin tiempo → **AutoML**.

Para **texto**:

- Pocos datos → **Gemini** / HF zero-shot.
- Dataset mediano → **embeddings** + **LightGBM**.

Regla de oro 💡

El **algoritmo** es el 20%. El otro 80% son **datos**, **validación honesta**, **métricas de negocio** y **despliegue** (todo el curso).

Documentación y recursos

- **scikit-learn** — scikit-learn.org/stable/supervised_learning.html (lineales, KNN, NB, LDA, SVM, árboles, ensembles, MLP)
- **XGBoost** — xgboost.readthedocs.io · **LightGBM** — lightgbm.readthedocs.io · **CatBoost** — catboost.ai
- **TabPFN** — github.com/PriorLabs/TabPFN · **AutoGluon** — auto.gluon.ai · **FLAML** — github.com/microsoft/FLAML
- **TabNet** — github.com/dreamquark-ai/tabnet · **rtdl (FT-Transformer)** — github.com/yandex-research/rtdl
- **Google**: TF Decision Forests — tensorflow.org/decision_forests · Vertex AI — cloud.google.com/vertex-ai · Gemini — ai.google.dev
- **Hugging Face** — huggingface.co · **Sentence-Transformers** — sbert.net

 Cada algoritmo del curso tiene su notebook de ejemplo en `02_Contenido/Mxx_*/notebooks/`.

¿Preguntas?

 jrodriguezm216@gmail.com

 jordandataexpert.com

 github.com/JordanKingPeru